Lecture 3: Multithreaded Programming
Operating Systems – EDA093/DIT401

Vincenzo Gulisano
vincenzo.gulisano@chalmers.se

UNIVERSITY OF
GOTHENBURG

What to read (main textbook)

- Chapter 2.2, 10.3.3*

*Some concepts will be covered later on (e.g., Copy-on-Write)

(extra facultative reading: 4.1-4.3, 4.4.1, 4.5-4.6 from Silberschatz Operating System Concepts)

Objectives

- To introduce the notion of a thread—a fundamental unit of CPU utilization that forms the basis of multithreaded computer systems
- To discuss the APIs for the Pthreads
- To explore several strategies that provide implicit threading
- To examine issues related to multithreaded programming

AGENDA

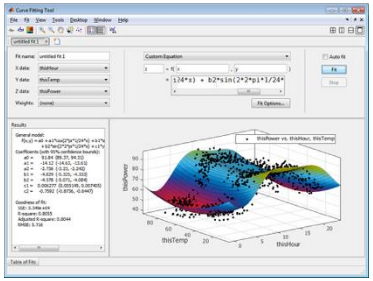
- Threads (Introduction)
- Multithreading models
- Implicit threading
- Threading issues

AGENDA

- Threads (Introduction)
- Multithreading models
- Implicit threading
- Threading issues

1. We run several programs at the same time

2. One CPU can only run one program at the time



Concurrent vs Parallel execution

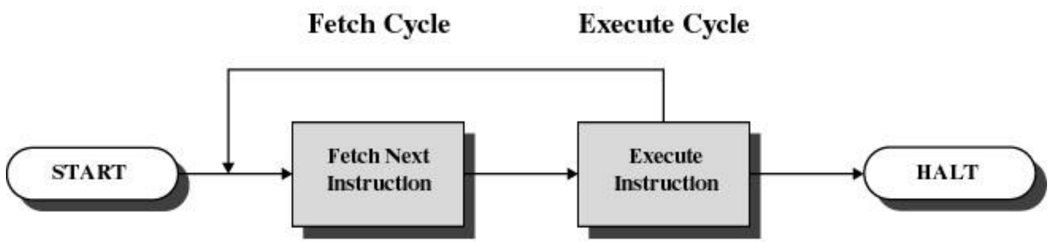
1. ~~We run several programs at the same time~~ → 1. We feel several programs run at the same time

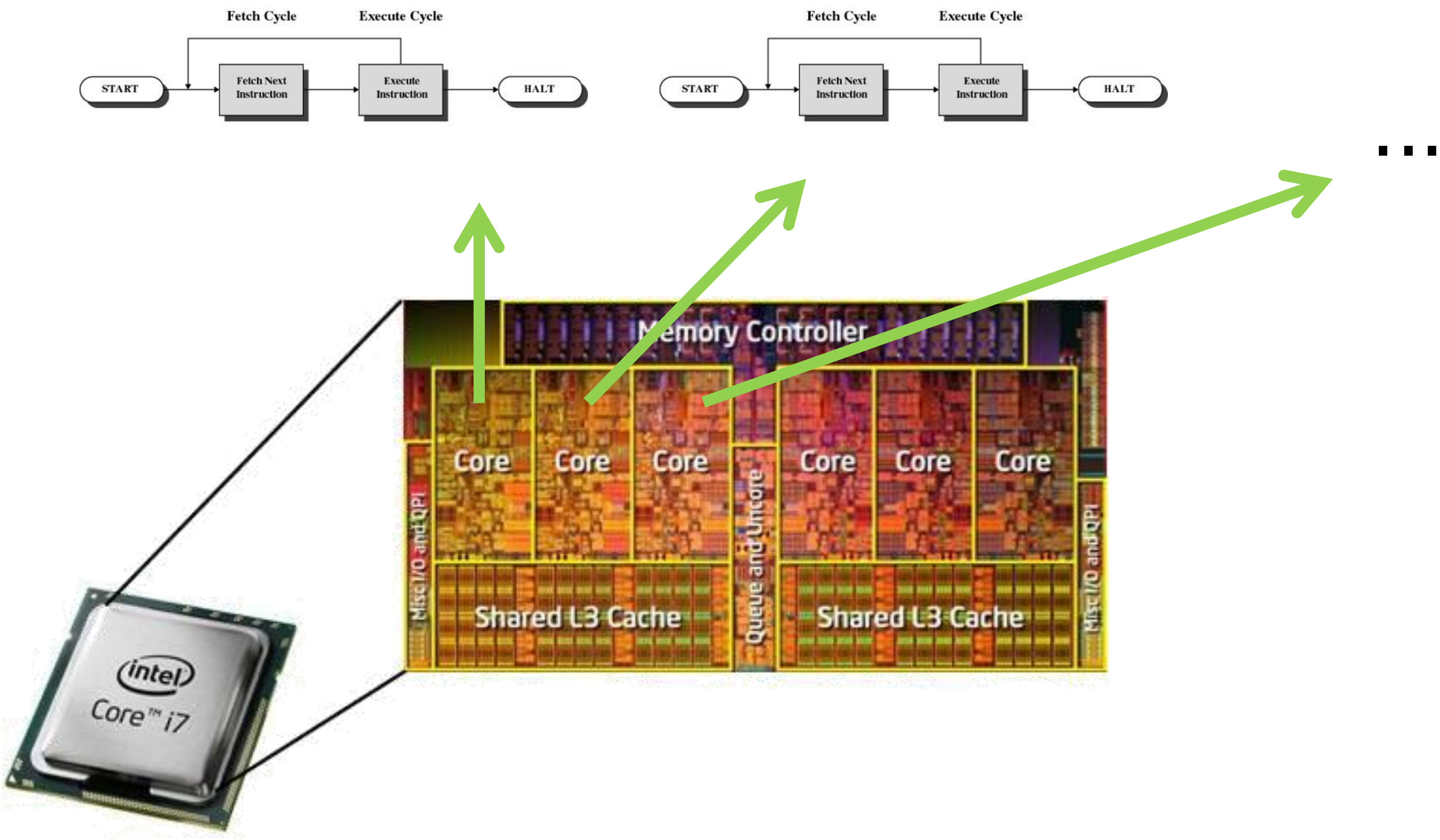
(Previous lecture)

2. ~~One CPU can only run one program at the time~~ → 2. Each CPU core can only run one program at the time

(This lecture)

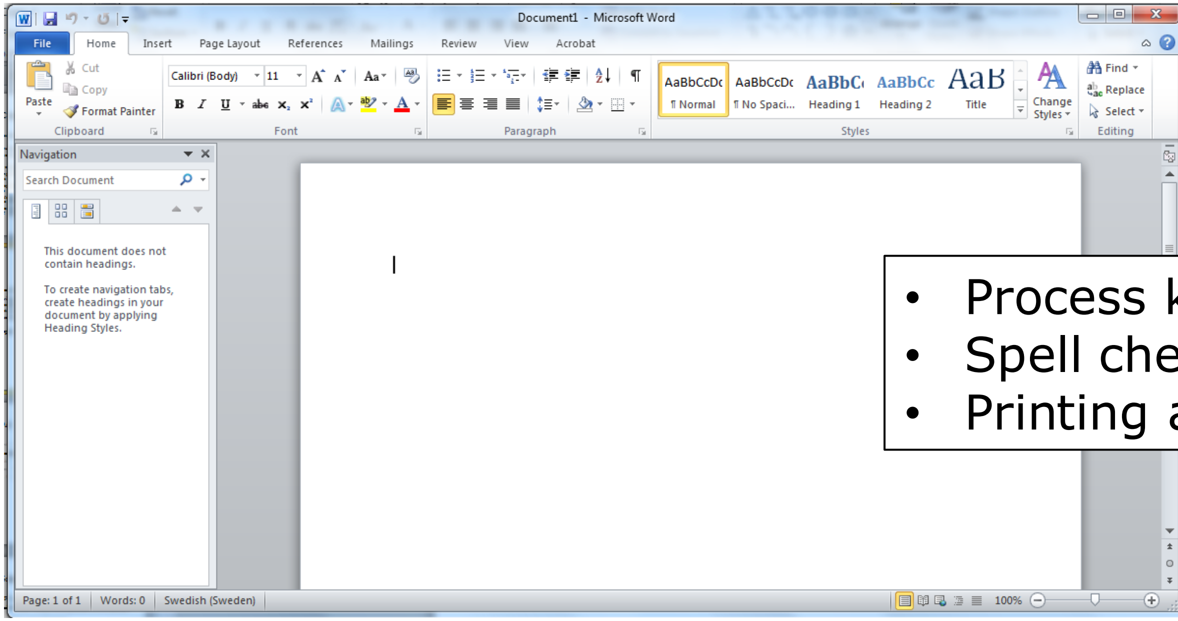
The basic CPU cycle



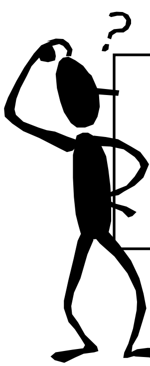


Fetch Cycle
Execute Cycle
START
Fetch Next Instruction
Execute Instruction
HALT
Fetch Cycle
Execute Cycle
START
Fetch Next Instruction
Execute Instruction
HALT
...
Memory Controller
Core
Core
Core
Core
Core
Core
Misc I/O and QPI
Queue and Uncore
Misc I/O and QPI
Shared L3 Cache
Shared L3 Cache
intel
Core™ i7

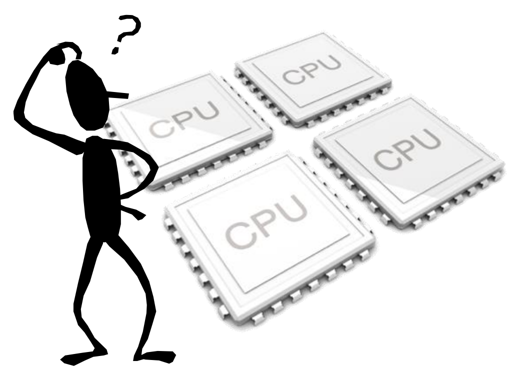
Discussion: parallel/concurrent execution of processes



- Process keyboard input…
- Spell checker…
- Printing a document…



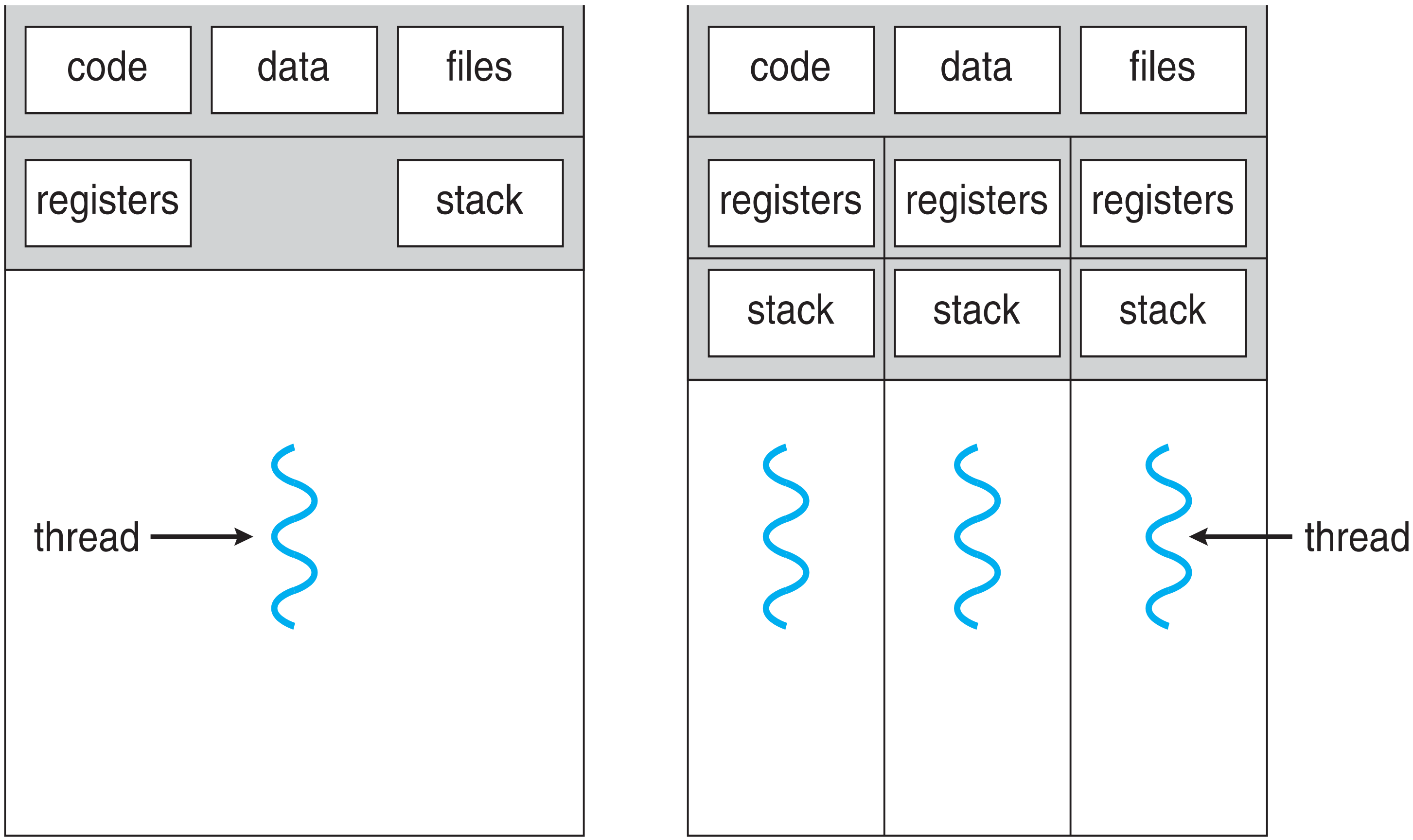
Important: only 1 process can be **running** on any processor at any instant.



Threads

- One process → multiple threads of execution
- Consider having multiple program counters per process
  - Multiple locations can execute at once
  - Multiple threads of control -> threads
- Must then have storage for thread details, multiple program counters in PCB

Single and Multithreaded Processes



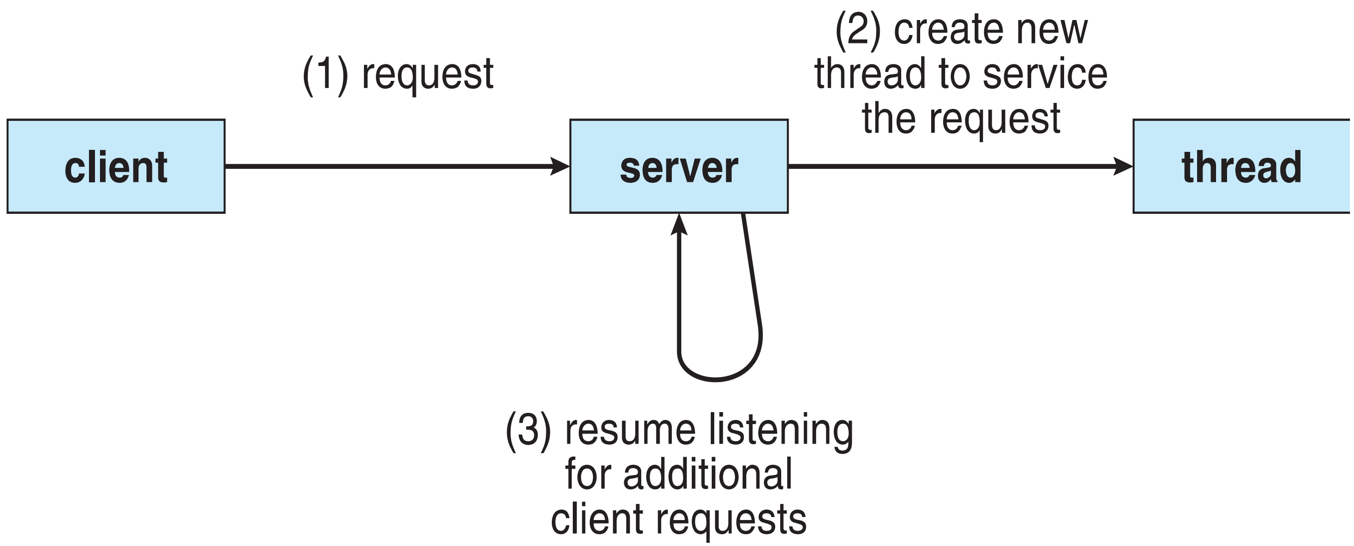
single-threaded process

multithreaded process

Motivation

- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
  - Update display
  - Fetch data
  - Spell checking
  - Answer a network request
- <u>Process creation is heavy-weight while thread creation is light-weight</u>
- <u>Can simplify code, increase efficiency</u>
- Kernels are generally multithreaded

Multithreaded Server Architecture



Benefits

- **Responsiveness –** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing
- **Economy –** cheaper than process creation, thread switching lower overhead than context switching
- **Scalability –** process can take advantage of multiprocessor architectures

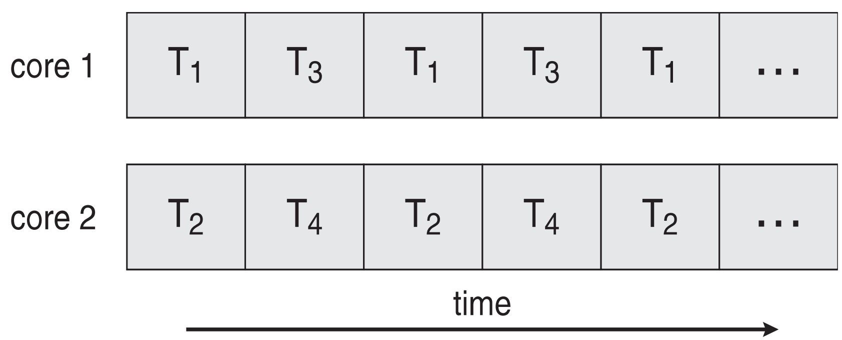
Multicore Programming

- **Multicore** or **multiprocessor** systems putting pressure on programmers, challenges include:
  - **Dividing activities**
  - **Balance**
  - **Data splitting**
  - **Data dependency**
  - **Testing and debugging**
- ***Parallelism*** implies a system can perform more than one task simultaneously
- ***Concurrency*** supports more than one task making progress
  - Single processor / core, scheduler providing concurrency

Concurrency vs. Parallelism

Concurrent execution on single-core system:

Parallelism on a multi-core system:

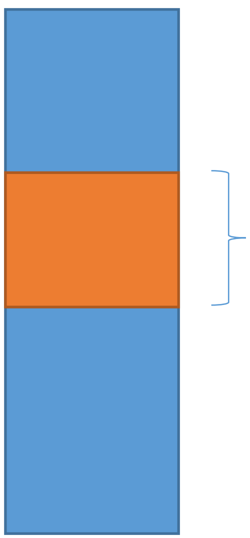


Multicore Programming (Cont.)

- Types of parallelism
  - **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each
  - **Task parallelism** – distributing threads across cores, each thread performing unique operation
- As # of threads grows, so does architectural support for threading
  - CPUs have cores as well as ***hardware threads***
  - Consider Oracle SPARC T4 with 8 cores, and 8 hardware threads per core

Question

- We have a program:



Serial portion S
(25% of the program)

How many times (X) faster?

$1 < X \leq 10$

$10 < X \leq 30$

$30 < X \leq 60$

$60 < X \leq 100$

1 core → 100 cores

Amdahl's Law

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- *S* is serial portion
- *N* processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As *N* approaches infinity, speedup approaches 1 / *S*

Serial portion of an application has disproportionate effect on performance gained by adding additional cores

AGENDA

- Threads (Introduction)
- Multithreading models
- Implicit threading
- Threading issues

User Threads and Kernel Threads

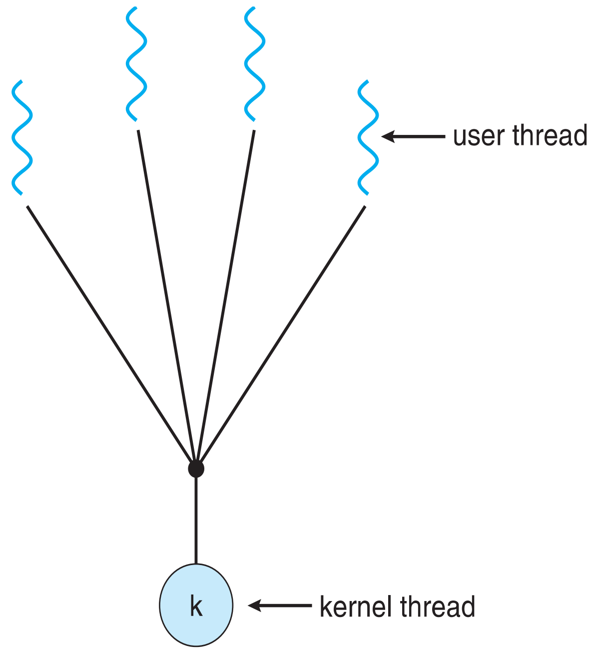
- **User threads** - management done by user-level threads library
- Three primary thread libraries:
  - POSIX **Pthreads**
  - Windows threads
  - Java threads
- **Kernel threads** - Supported by the Kernel
- Examples – virtually all general purpose operating systems, including:
  - Windows
  - Solaris
  - Linux
  - Tru64 UNIX
  - Mac OS X

Multithreading Models

- Many-to-One
- One-to-One
- Many-to-Many

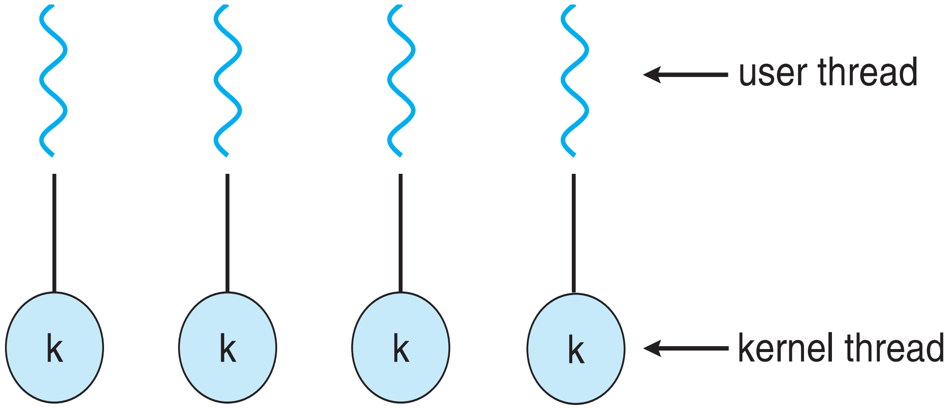
Many-to-One

- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on muticore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
    - **Solaris Green Threads**
    - **GNU Portable Threads**



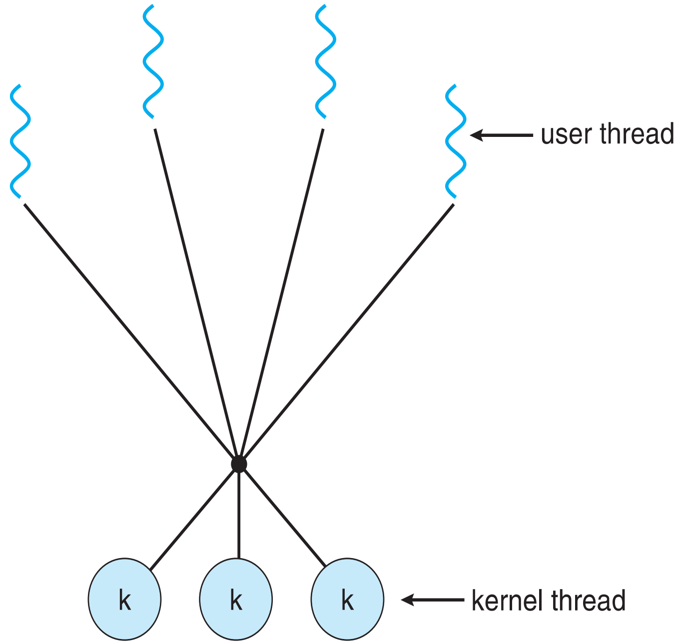
One-to-One

- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
  - Windows
  - Linux
  - Solaris 9 and later



Many-to-Many Model

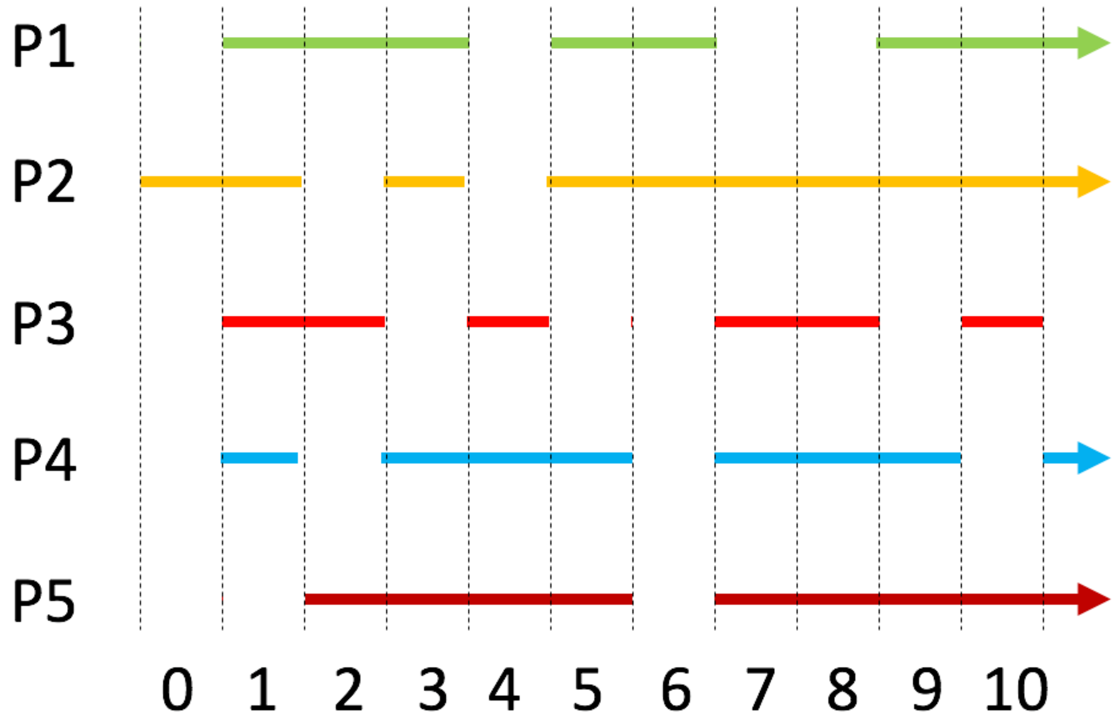
- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Solaris prior to version 9
- Windows with the *ThreadFiber* package



Questions - Part 1

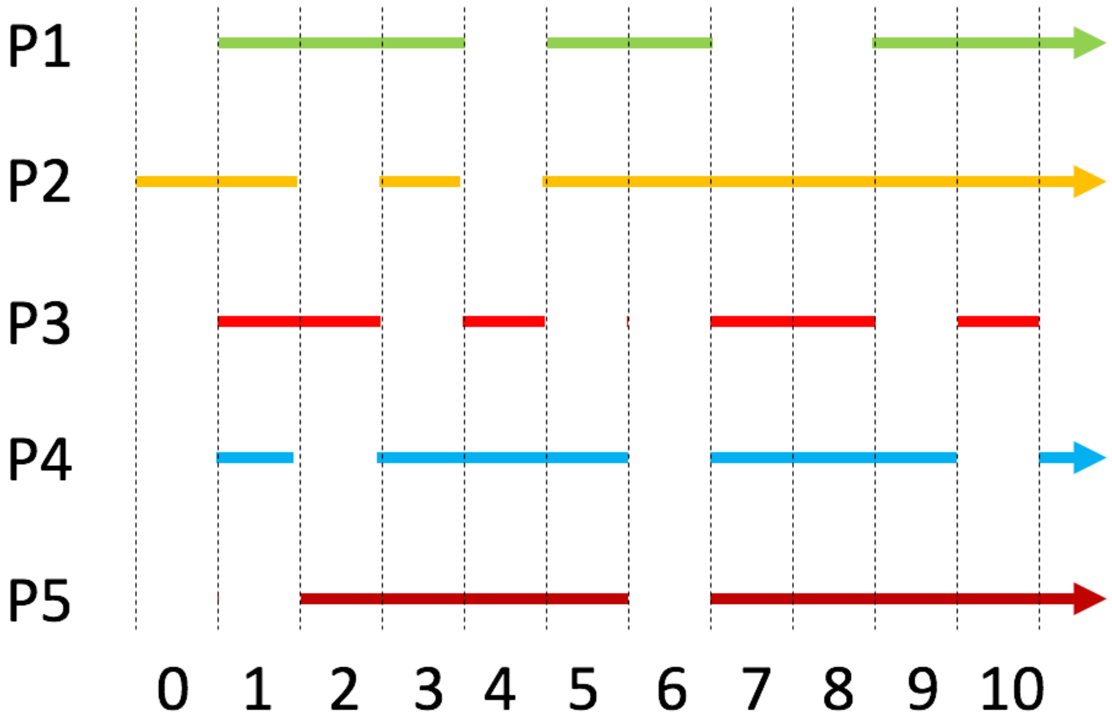
Given the processes running in the figure, how many cores does the CPU has?

1. exactly 4
2. at least 4
3. no more than 4
4. cannot say



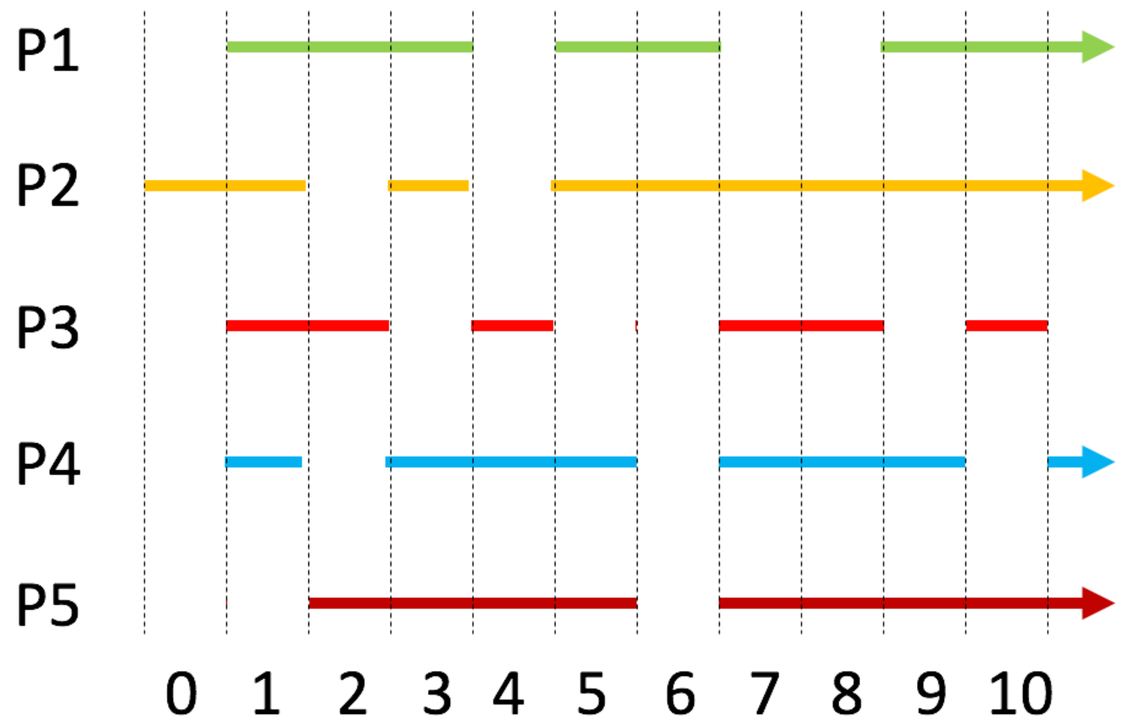
Given the processes running in the figure, why P2 gets switched at slot 2?

1. To run another process
2. cannot say



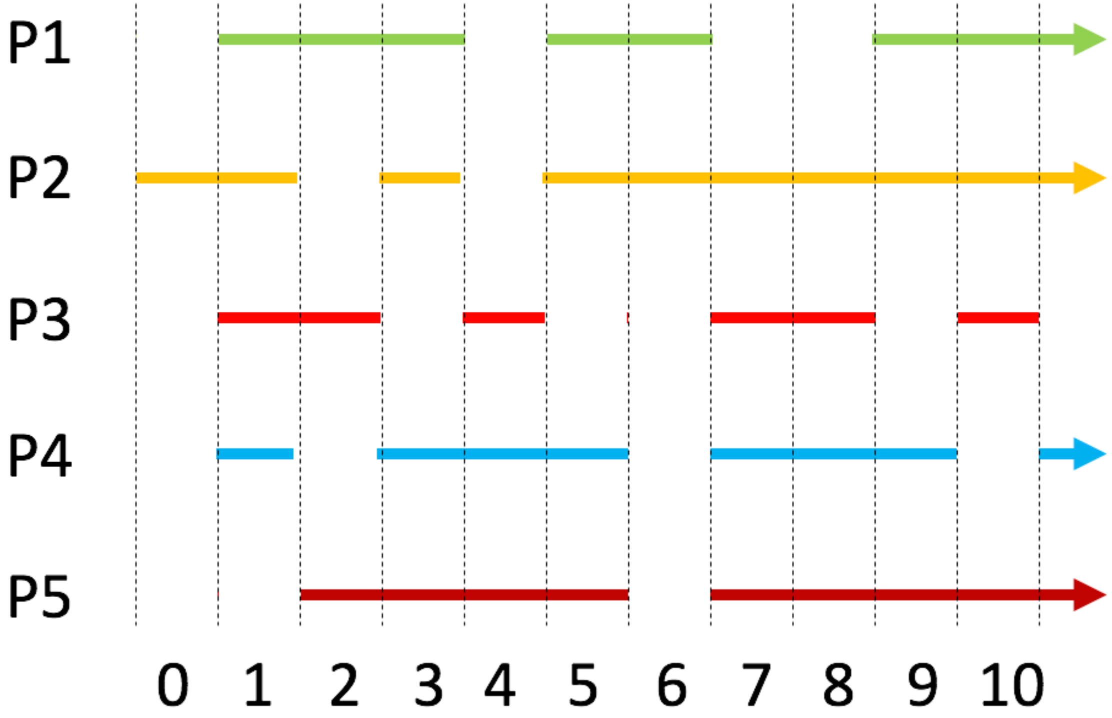
Given the processes running in the figure, why P3 gets switched at slot 5?

1. To run another process
2. cannot say



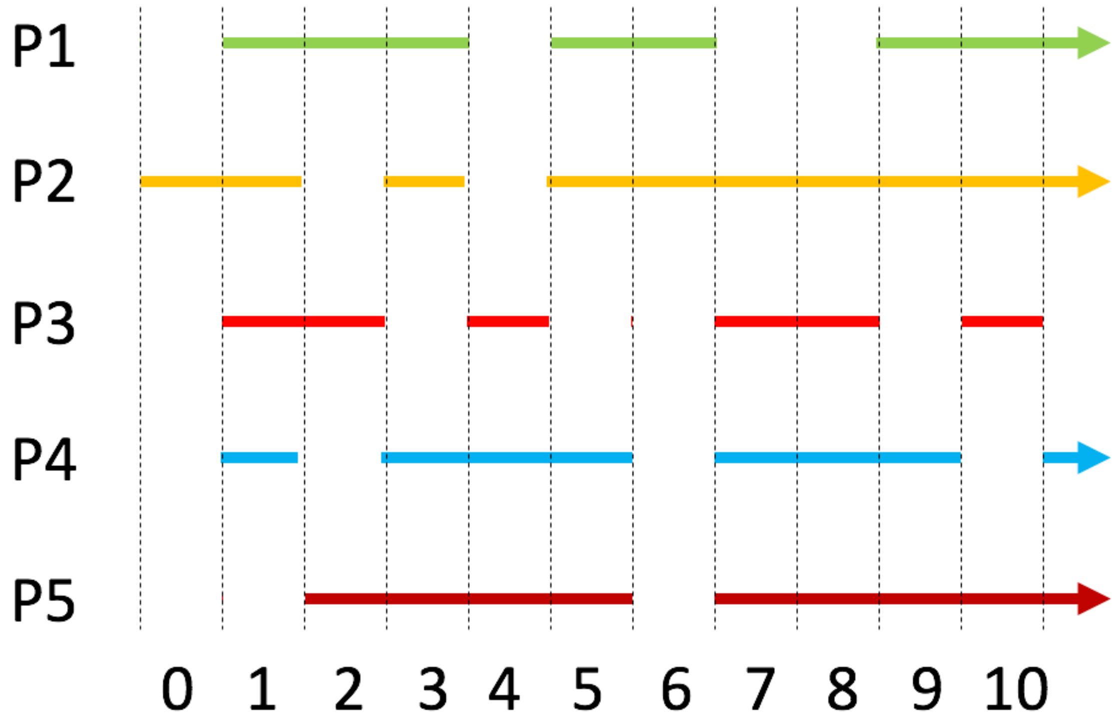
Given the processes running in the figure, do P2 and P3 run in parallel?

1. Yes
2. No
3. Cannot say



Given the processes running in the figure, do P2 and P3 run concurrently?

1. Yes
2. No
3. Cannot say



Thread Libraries

- **Thread library** provides programmer with API for creating and managing threads
- Two primary ways of implementing
  - Library entirely in user space
  - Kernel-level library supported by the OS

Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- ***Specification***, not ***implementation***
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Solaris, Linux, Mac OS X)

Pthreads Example

Summing 0 … n

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
  pthread_t tid; /* the thread identifier */
  pthread_attr_t attr; /* set of thread attributes */

  if (argc != 2) {
    fprintf(stderr,"usage: a.out <integer value>\n");
    return -1;
  }
  if (atoi(argv[1]) < 0) {
    fprintf(stderr,"%d must be >= 0\n",atoi(argv[1]));
    return -1;
  }
```

Pthreads Example (Cont.)

```
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid,&attr,runner,argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid,NULL);

    printf("sum = %d\n",sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```

Pthreads Code for Joining 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
   pthread_join(workers[i], NULL);
```

AGENDA

- Threads (Introduction)
- Multithreading models
- Implicit threading
- Threading Issues

Implicit Threading

- Growing in popularity as numbers of threads increase, program correctness more difficult with explicit threads
- Creation and management of threads done by compilers and run-time libraries rather than programmers
- Two methods explored
  - Thread Pools
  - OpenMP

Thread Pools

- Create a number of threads in a pool where they await work
- Advantages:
  - Usually slightly faster to service a request with an existing thread than create a new thread
  - Allows the number of threads in the application(s) to be bound to the size of the pool
  - Separating task to be performed from mechanics of creating task allows different strategies for running task
    - i.e.Tasks could be scheduled to run periodically

OpenMP

- Set of compiler directives and an API for C, C++, FORTRAN
- Provides support for parallel programming in shared-memory environments
- Identifies **parallel regions** – blocks of code that can run in parallel

```
#pragma omp parallel
```

Create as many threads as there are cores

```
#pragma omp parallel for
  for(i=0;i<N;i++) {
     c[i] = a[i] + b[i];
}
```

Run for loop in parallel

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
   /* sequential code */

   #pragma omp parallel
   {
      printf("I am a parallel region.");
   }

   /* sequential code */

   return 0;
}
```

AGENDA

- Threads (Introduction)
- Multithreading models
- Implicit threading
- Threading Issues

Threading Issues

- Semantics of **fork()** and **exec()** system calls
- Signal handling
  - Synchronous and asynchronous
- Thread cancellation of target thread
  - Asynchronous or deferred

C Program Forking Separate Process

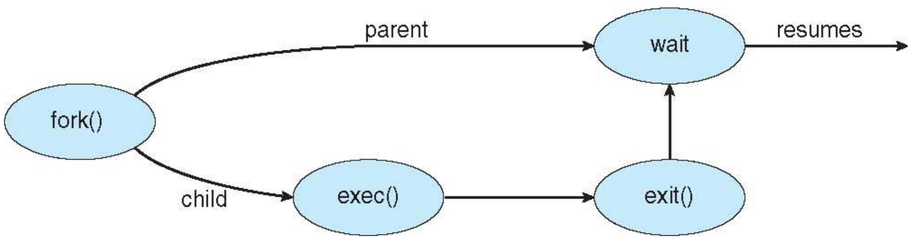
```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```



Semantics of fork() and exec()

- Does **`fork()`** duplicate only the calling thread or all threads?
  - Some UNIXes have two versions of fork
- **`exec()`** usually works as normal – replace the running process including all threads

Signal Handling

- Signals are used in UNIX systems to notify a process that a particular event has occurred.
- A signal handler is used to process signals
  - Signal is generated by particular event
  - Signal is delivered to a process
  - Signal is handled by one of two signal handlers:
    - default
    - user-defined
- Every signal has default handler that kernel runs when handling signal
  - User-defined signal handler can override default
  - For single-threaded, signal delivered to process

Signal Handling (Cont.)

- Where should a signal be delivered for multi-threaded?
  - Deliver the signal to the thread to which the signal applies
  - Deliver the signal to every thread in the process
  - Deliver the signal to certain threads in the process
  - Assign a specific thread to receive all signals for the process

Thread Cancellation

- Terminating a thread before it has finished
- Thread to be canceled is **target thread**
- Two general approaches:
  - **Asynchronous cancellation** terminates the target thread immediately
  - **Deferred cancellation** allows the target thread to periodically check if it should be cancelled
- Pthread code to create and cancel a thread:

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

. . .

/* cancel the thread */
pthread_cancel(tid);
```

Thread Cancellation (Cont.)

- Invoking thread cancellation requests cancellation, but actual cancellation depends on thread state

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

- If thread has cancellation disabled, cancellation remains pending until thread enables it
- Default type is deferred
  - Cancellation only occurs when thread reaches **cancellation point**
    - I.e. **`pthread_testcancel()`**
    - Then **cleanup handler** is invoked
- On Linux systems, thread cancellation is handled through signals

Questions - Part 2

Given the code, the variable sum is not shared between threads.

1. True
2. False
3. cannot say

```c
int sum;

int main() {
    [...]
    pthread_create(&tid,&attr,runner,argv[1]);
    pthread_join(tid,NULL);
    ptrintf("sum = %d\n",sum);
    [...]
}

void *runner(void *param) {
    int i, upper = atoi(param);
    sum=0;
    for(i = 1; i <= upper; i++)
        sum += i;
    pthread_exit(0);
}
```

Given the code, the main function can complete before the created thread completes.

1. False
2. True

```c
int sum;

int main() {
    [...]
    pthread_create(&tid,&attr,runner,argv[1]);
    pthread_join(tid,NULL);
    ptrintf("sum = %d\n",sum);
    [...]
}

void *runner(void *param) {
    int i, upper = atoi(param);
    sum=0;
    for(i = 1; i <= upper; i++)
        sum += i;
    pthread_exit(0);
}
```

Given the code, pthread_join might find the created thread has already completed when run.

1. True
2. False

```c
int sum;

int main() {
    [...]
    pthread_create(&tid,&attr,runner,argv[1]);
    pthread_join(tid,NULL);
    ptrintf("sum = %d\n",sum);
    [...]
}

void *runner(void *param) {
    int i, upper = atoi(param);
    sum=0;
    for(i = 1; i <= upper; i++)
        sum += i;
    pthread_exit(0);
}
```

Given the code, the final value of sum can change at different executions.

1. Yes
2. No
3. The question is ambiguous

```c
int sum;

int main() {
    [...]
    pthread_create(&tid,&attr,runner,argv[1]);
    pthread_join(tid,NULL);
    ptrintf("sum = %d\n",sum);
    [...]
}

void *runner(void *param) {
    int i, upper = atoi(param);
    sum=0;
    for(i = 1; i <= upper; i++)
        sum += i;
    pthread_exit(0);
}
```